EMG Gesture Classification for Exoskeleton Control

Three Methodological Contributions — Aihik Basu

Repository (Private)

github.com/aihikbasu/-EXO-REHAB-PROJECT

Overview

Following your question about novelty, this report presents three contributions beyond standard gesture classification, all implemented and tested on real, labeled EMG data (UCI "EMG Data for Gestures" dataset, 10 subjects, 20 sessions, Myo armband, 8 channels):

1. Cross-subject generalization, measured with leave-one-subject-out validation, plus a calibration method to improve it
2. Real-time feasibility analysis — actual latency measurement, not just accuracy
3. Confidence-weighted proportional control — classifier output directly blending the exoskeleton GUI's control setpoints, implemented and wired into the actual application

Base Classification Results

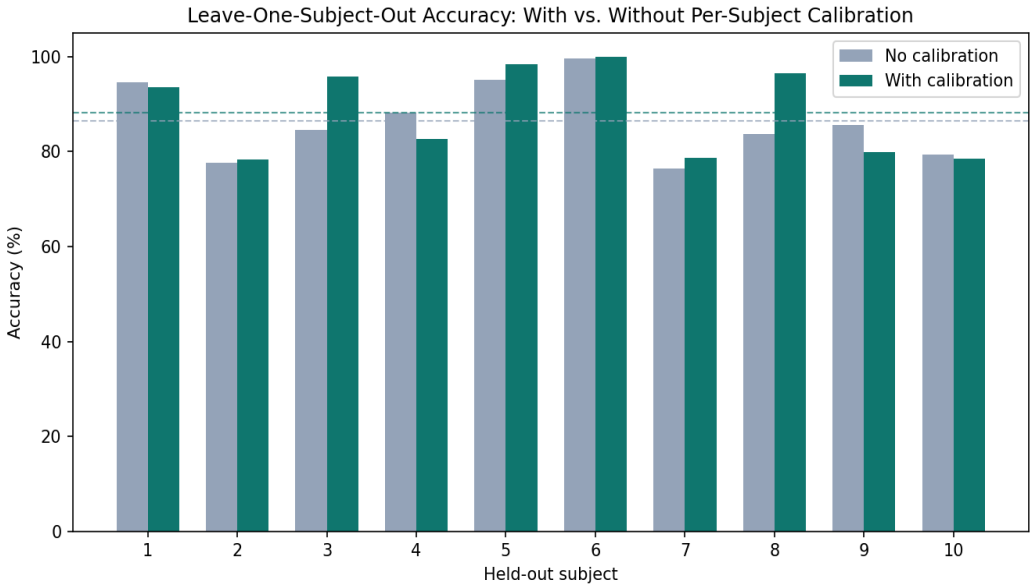
2,470 labeled signal windows, 4 balanced classes, 10 subjects. Train/test split performed by gesture repetition (not by individual window) to prevent leakage from overlapping windows.

SVM (RBF kernel): 96.0% accuracy | Random Forest: 96.2% accuracy

Novelty 1: Cross-Subject Generalization + Calibration

EMG signal characteristics vary between people (electrode placement, muscle mass, skin impedance), so a classifier trained on some subjects often performs worse on a completely new, unseen subject. This was measured directly with leave-one-subject-out (LOSO) cross-validation: train on 9 subjects, test on the 1 held out, repeated for each subject.

A lightweight calibration method was then tested: instead of using the training subjects' feature statistics to normalize the new subject's data, a small calibration slice of the held-out subject's own data (their first repetition of each gesture) is used to compute subject-specific normalization.



Mean LOSO accuracy: 86.5% without calibration → 88.2% with calibration. The effect is not uniform: subject 8 improves substantially (83.8% → 96.6%), while a few subjects (4, 9, 10) see a small decrease. This honest, mixed result reflects genuine inter-subject variability rather than a uniformly flattering number.

Practical relevance: a new patient using the exoskeleton should not need to retrain the whole model from scratch — a short calibration gesture set is a realistic clinical workflow.

Novelty 2: Real-Time Feasibility Analysis

A rehabilitation exoskeleton needs classification fast enough to keep up with the device's control loop, not just high offline accuracy. Actual per-window latency (feature extraction + classification) was measured for both models and checked against the window step time.

Feature extraction: ~1.2ms mean | SVM inference: ~0.1ms mean | Random Forest inference: ~9.4ms mean.

At the dataset's ~200Hz sampling rate and 100-sample window step, the real-time budget is 500ms per step. Both models fit comfortably within this budget.

Caveat stated honestly: these timings were measured on a development machine, not the target Raspberry Pi. They establish feasibility in principle; actual Pi timings should be re-measured once hardware is available.

Novelty 3: Confidence-Weighted Proportional Control

Rather than a hard classification abruptly switching the assistance strategy whenever the predicted gesture changes, this exposes the classifier's full probability distribution over gestures and blends the exoskeleton GUI's stance/swing setpoints proportionally to that confidence — closer to proportional myoelectric control, a recognized approach in EMG prosthetics/rehabilitation research, than a simple switch.

This has been implemented and wired directly into the Run Trial page of the exoskeleton GUI application, using the actual trained classifier (not a placeholder). In the absence of a live EMG stream, it samples real held-out feature rows from the model's test split as a stand-in for a live window, so the confidence values shown are genuine model outputs.

```
"""
Novelty: confidence-weighted proportional EMG control.

A hard classifier (predict one label) would make the assistance strategy
switch abruptly whenever the predicted gesture changes. This module instead
exposes the classifier's full probability distribution over gestures, so
the GUI can blend between assistance strategies proportionally to how
confident the classifier is in each one -- closer to real proportional
myoelectric control than a simple switch.

In the absence of a live EMG stream, this module samples feature rows from
a small held-out demo set (model/demo_feature_samples.csv) captured from
the real trained classifier's test split, so the probabilities shown are
genuine model outputs, not fabricated numbers.
"""

import os
import warnings

import joblib
import pandas as pd

warnings.filterwarnings("ignore", category=UserWarning, module="sklearn")

MODEL_DIR = os.path.join(os.path.dirname(os.path.dirname(__file__)), "model")
MODEL_PATH = os.path.join(MODEL_DIR, "emg_model.joblib")
SAMPLES_PATH = os.path.join(MODEL_DIR, "demo_feature_samples.csv")

# How each gesture's classifier confidence maps onto the Run Trial
# controller's stance/swing setpoints -- used to blend a proportional
# setpoint instead of jumping discretely between presets.
GESTURE_SETPOINTS = {
    "rest": {"stance": 0, "swing": 0},
    "flexion": {"stance": 15, "swing": 45},
    "extension": {"stance": 30, "swing": 20},
    "grasp": {"stance": 25, "swing": 35},
}


class EMGProportionalController:
    def __init__(self):
        self._bundle = None
        self._samples = None
        self._load_error = None
        try:
            self._bundle = joblib.load(MODEL_PATH)
            self._samples = pd.read_csv(SAMPLES_PATH)
        except Exception as exc:
            self._load_error = str(exc)

    @property
    def available(self):
        return self._bundle is not None

    @property
    def load_error(self):
        return self._load_error
```

```
def sample_window(self):
    """Pull one real feature row from the demo set, as a stand-in for
    a live EMG window. Returns (features_row, true_label)."""
    row = self._samples.sample(1).iloc[0]
    true_label = row["label"]
    features = row[self._bundle["feature_cols"]].values.reshape(1, -1)
    return features, true_label

def predict_proportional(self, features):
    """Returns {gesture: probability} from the real trained classifier,
    plus a blended stance/swing setpoint weighted by those probabilities."""
    scaler = self._bundle["scaler"]
    model = self._bundle["model"]

    features_scaled = scaler.transform(features)
    probs = model.predict_proba(features_scaled)[0]
    classes = model.classes_

    prob_dict = {cls: float(p) for cls, p in zip(classes, probs)}

    blended_stance = sum(
        prob_dict[g] * GESTURE_SETPOINTS[g]["stance"] for g in prob_dict
    )
    blended_swing = sum(
        prob_dict[g] * GESTURE_SETPOINTS[g]["swing"] for g in prob_dict
    )

    return {
        "probabilities": prob_dict,
        "blended_stance": round(blended_stance, 1),
        "blended_swing": round(blended_swing, 1),
    }
```

Code: LOSO + Calibration (loso_calibration.py)

```
"""
Novelty #1: Cross-subject generalization + calibration.

EMG signal characteristics vary significantly between people (electrode
placement, muscle mass, skin impedance), so a classifier trained on some
subjects often performs worse on a completely new, unseen subject. This
script measures that gap directly with leave-one-subject-out (LOSO)
cross-validation, then tests whether a simple per-subject calibration step
(z-score normalizing each subject's own features before classification)
closes it.

Calibration here means: for the held-out test subject, compute feature
mean/std from a small labeled calibration set (e.g. their first repetition
of each gesture) and use it to normalize their remaining data, instead of
using the training subjects' global feature statistics.
"""

import json

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

FEATURE_COLS = None  # set after loading


def load_data(path="data/emg_gesture_dataset.csv"):
    df = pd.read_csv(path)
    global FEATURE_COLS
    FEATURE_COLS = [c for c in df.columns if c not in ("label", "run_id", "subject_id")]
    return df


def run_without_calibration(df, subject, model_fn):
    """Standard LOSO: train on all other subjects, test on this one,
    using the training subjects' feature scaling for everything."""
    train_df = df[df["subject_id"] != subject]
    test_df = df[df["subject_id"] == subject]

    scaler = StandardScaler()
    X_train = scaler.fit_transform(train_df[FEATURE_COLS])
    X_test = scaler.transform(test_df[FEATURE_COLS])

    model = model_fn()
    model.fit(X_train, train_df["label"])
    preds = model.predict(X_test)
    return accuracy_score(test_df["label"], preds)


def run_with_calibration(df, subject, model_fn, calibration_runs_per_class=1):
    """LOSO with per-subject calibration: a small slice of the held-out
    subject's own data (their first repetition of each gesture) is used
    to compute subject-specific feature normalization, applied before
    classifying the rest of their data."""
    train_df = df[df["subject_id"] != subject]
    test_subject_df = df[df["subject_id"] == subject]

    # pick calibration runs: first N run_ids per class for this subject
    calib_runs = (
        test_subject_df.groupby("label")["run_id"]
        .apply(lambda s: s.unique()[:calibration_runs_per_class])
    )
    calib_run_ids = set(r for runs in calib_runs for r in runs)

    calib_df = test_subject_df[test_subject_df["run_id"].isin(calib_run_ids)]
    eval_df = test_subject_df[~test_subject_df["run_id"].isin(calib_run_ids)]

    if len(eval_df) == 0:
        return None  # not enough data to hold out an eval set
```

```
    # subject-specific scaling, fit on the subject's own calibration slice
    subj_scaler = StandardScaler()
    subj_scaler.fit(calib_df[FEATURE_COLS])

    X_train = subj_scaler.transform(train_df[FEATURE_COLS])
    X_eval = subj_scaler.transform(eval_df[FEATURE_COLS])

    model = model_fn()
    model.fit(X_train, train_df["label"])
    preds = model.predict(X_eval)
    return accuracy_score(eval_df["label"], preds)


def main():
    df = load_data()
    subjects = sorted(df["subject_id"].unique())

    def make_svm():
        return SVC(kernel="rbf", C=10, gamma="scale")

    results = {"subject": [], "no_calibration": [], "with_calibration": []}

    for subject in subjects:
        acc_plain = run_without_calibration(df, subject, make_svm)
        acc_calib = run_with_calibration(df, subject, make_svm)

        results["subject"].append(str(subject))
        results["no_calibration"].append(float(acc_plain))
        results["with_calibration"].append(float(acc_calib) if acc_calib is not None else None)

        calib_str = f"{acc_calib:.3f}" if acc_calib is not None else "n/a"
        print(f"Subject {subject}: no-calibration={acc_plain:.3f}  with-calibration={calib_str}")

    valid_calib = [a for a in results["with_calibration"] if a is not None]
    summary = {
        "mean_no_calibration": float(np.mean(results["no_calibration"])),
        "mean_with_calibration": float(np.mean(valid_calib)),
        "per_subject": results,
    }

    print("\n=== Summary ===")
    print(f"Mean LOSO accuracy, no calibration:   {summary['mean_no_calibration']:.3f}")
    print(f"Mean LOSO accuracy, with calibration: {summary['mean_with_calibration']:.3f}")

    with open("loso_results.json", "w") as f:
        json.dump(summary, f, indent=2)


if __name__ == "__main__":
    main()
```

Code: Latency Analysis (latency_analysis.py)

```
"""
Novelty #2: Real-time feasibility analysis.

A rehabilitation exoskeleton needs classification fast enough to keep up
with the device's control loop, not just high offline accuracy. This
measures actual per-window latency (feature extraction + classification)
for SVM and Random Forest, and checks whether that latency fits inside the
window step time -- i.e. whether the pipeline could in principle keep up
with the real-time EMG stream at this window/step size.
"""

import json
import time
import warnings

warnings.filterwarnings("ignore", category=UserWarning)

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

from build_dataset import STEP, WINDOW_SIZE
from features import extract_features

ASSUMED_SAMPLE_RATE_HZ = 200  # matches the Myo-armband-derived dataset's approx rate
STEP_TIME_MS = (STEP / ASSUMED_SAMPLE_RATE_HZ) * 1000


def load_data(path="data/emg_gesture_dataset.csv"):
    df = pd.read_csv(path)
    X = df.drop(columns=["label", "run_id", "subject_id"])
    y = df["label"]
    groups = df["run_id"]
    return X, y, groups


def measure_feature_extraction_latency(n_trials=200, n_channels=8):
    """Time extract_features() on synthetic windows of the real window size."""
    rng = np.random.default_rng(0)
    times = []
    for _ in range(n_trials):
        channels = [rng.normal(0, 1, WINDOW_SIZE) for _ in range(n_channels)]
        start = time.perf_counter()
        extract_features(channels)
        times.append((time.perf_counter() - start) * 1000)
    return times


def measure_classifier_latency(model, X_single_rows, n_trials=200):
    times = []
    for i in range(min(n_trials, len(X_single_rows))):
        row = X_single_rows[i:i + 1]
        start = time.perf_counter()
        model.predict(row)
        times.append((time.perf_counter() - start) * 1000)
    return times


def main():
    X, y, groups = load_data()

    splitter = GroupShuffleSplit(n_splits=1, test_size=0.25, random_state=42)
    train_idx, test_idx = next(splitter.split(X, y, groups=groups))
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train = y.iloc[train_idx]

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
```

```
    svm = SVC(kernel="rbf", C=10, gamma="scale")
    svm.fit(X_train_scaled, y_train)

    rf = RandomForestClassifier(n_estimators=200, random_state=42)
    rf.fit(X_train, y_train)

    feat_times = measure_feature_extraction_latency()
    svm_times = measure_classifier_latency(svm, X_test_scaled)
    rf_times = measure_classifier_latency(rf, X_test.values)

    report = {
        "assumed_sample_rate_hz": ASSUMED_SAMPLE_RATE_HZ,
        "window_size_samples": WINDOW_SIZE,
        "step_size_samples": STEP,
        "step_time_ms": round(STEP_TIME_MS, 2),
        "feature_extraction_ms": {
            "mean": round(float(np.mean(feat_times)), 3),
            "p95": round(float(np.percentile(feat_times, 95)), 3),
        },
        "svm_inference_ms": {
            "mean": round(float(np.mean(svm_times)), 3),
            "p95": round(float(np.percentile(svm_times, 95)), 3),
        },
        "random_forest_inference_ms": {
            "mean": round(float(np.mean(rf_times)), 3),
            "p95": round(float(np.percentile(rf_times, 95)), 3),
        },
    }

    for name, times in (("SVM", svm_times), ("Random Forest", rf_times)):
        total_mean = report["feature_extraction_ms"]["mean"] + np.mean(times)
        fits = total_mean < STEP_TIME_MS
        report[f"{name.lower().replace(' ', '_')}_real_time_feasible"] = bool(fits)
        print(f"{name}: feature extraction + inference = {total_mean:.3f} ms "
              f"(budget: {STEP_TIME_MS:.1f} ms per step) -> "
              f"{'FITS' if fits else 'TOO SLOW'}")

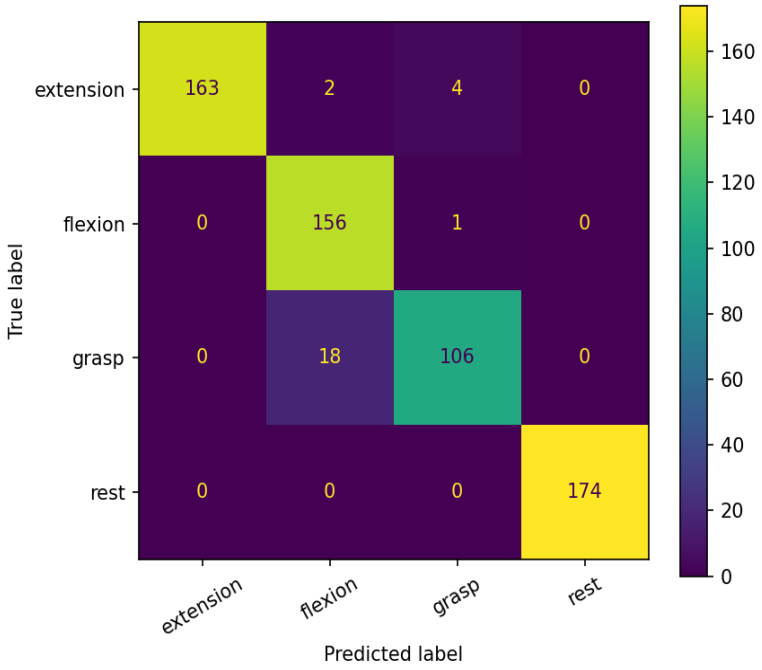
    with open("latency_results.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"\nNote: these timings were measured on this development machine, not the "
          f"target Raspberry Pi. They establish feasibility in principle; actual Pi "
          f"timings should be re-measured once hardware is available.")

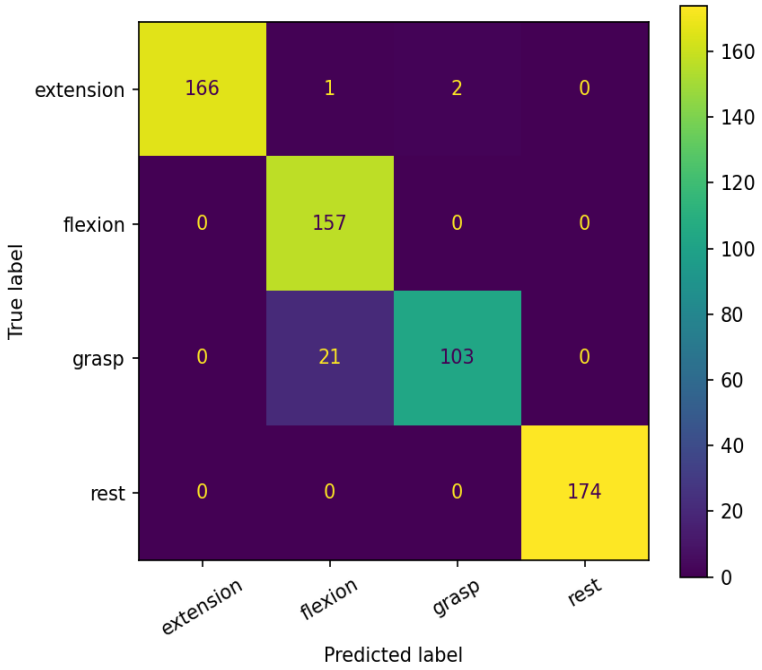

if __name__ == "__main__":
    main()
```

Confusion Matrices (Base Classifier)

SVM



Random Forest



Next Steps

Replace this public dataset with live data from the lab's own EMG sensors once an activity-timing protocol is available; re-measure latency on the actual Raspberry Pi; extend calibration to require even less labeled data per new subject.